

BUILDING MANAGER

Comprehensive Project Analysis

Version: 2.2.0

Date: July 2026

Type: Business & Technical Analysis

Prepared: Business Analysis Team

Table of Contents

1. Executive Summary
2. Project Overview
3. Technology Stack Analysis
4. Application Architecture
5. User Roles & Access Control
6. Functional Modules
7. API Integration & Backend Services
8. Data Models & State Management
9. Internationalization & Localization
10. Build, Deployment & DevOps
11. Quality Assurance & Testing
12. Security Analysis
13. Performance Considerations
14. Risk Assessment
15. Recommendations

1. Executive Summary

The **Building Manager** application is a full-featured Property & Building Management SaaS platform built on top of the Mantis Angular Dashboard Template by CodedThemes. It serves as a comprehensive digital solution for managing residential and commercial buildings, enabling property owners, property managers, building managers, residents, and administrators to collaborate through a unified interface.

Business Domain: Property Technology (PropTech) — Building & Community Management
Target Market: Building owners, property management companies, residential communities, commercial property managers
Delivery Model: Single-page application (SPA) served via Docker/Nginx with REST API backend

TECH STACK

Angular 20 + Bootstrap

VERSION

2.2.0

USER ROLES

8 Distinct Roles

LANGUAGES

5 Languages

API SERVICES

27 Backend Services

RUNTIME DEPENDENCIES

78 Packages

The application implements role-based access control with **eight distinct user roles**, supports **five languages** (Greek, English, French, Romanian, Chinese), and integrates **27 backend API services** covering authentication, building management, expense statements, payments, support tickets, professional directories, polls, and more.

Key Business Capabilities

- **Building & Apartment Management** — CRUD operations for buildings, apartments, and members

- **Common Expense Statements** — Create, allocate, and track shared building expenses
- **Payment Processing** — Track payments, balances, and collection rates per building
- **Support Ticket System** — Issue reporting, assignment, and resolution workflow
- **Professional Directory** — Searchable marketplace for property-related service providers
- **Community Features** — Polls, calendar events, announcements, chat
- **Role-Specific Dashboards** — Tailored analytics for admin, PM, building manager, and user
- **Theme Customization** — White-label ready with extensive UI theming options

Current Development Stage

The application appears to be in **late-stage development or early production**. Core functionality is implemented across all major modules, with unit tests for key authentication and support ticket pages. The project uses a pre-release version of Angular (20.0.0-next.8) and includes both unit testing (Jasmine/Karma) and E2E testing (Playwright) frameworks.

2. Project Overview

2.1 Identity & Origin

Property	Value
Package Name	mantis-angular-ng-bootstrap
Application Title	Building Manager
Version	2.2.0
Template Origin	Mantis Angular Dashboard Template by CodedThemes
Base Framework	Angular 20.0.0-next.8
UI Framework	Bootstrap 5.3.6 + ng-bootstrap 18.0.0
Index Title	Mantis Angular Dashboard Template
Root Selector	app-root

2.2 Project Size & Scope

Metric	Count
Runtime Dependencies	78
Dev Dependencies	22
Backend API Services	27
Data Models / Interfaces	42
Translation Languages	5
User Roles	8
Functional Modules	14+
Unit Test Spec Files	16
Third-Party UI Components	20+

2.3 Project Maturity Assessment

Dimension	Rating	Notes
Feature Completeness	High	All major modules implemented
Code Organization	High	Well-structured modular architecture
Test Coverage	Low-Medium	Only 16 spec files across limited modules
Documentation	Low	No README or formal docs in repo
State Management	Medium	Services + localStorage, no dedicated library
Security	High	JWT auth, role guards, interceptors in place
Build Pipeline	High	Docker, multi-stage build, Nginx config
i18n Readiness	High	5 languages with custom loader

3. Technology Stack Analysis

3.1 Core Framework

The application is built on **Angular 20.0.0-next.8**, a pre-release version of the Angular framework. This represents both an opportunity (early access to latest features) and a risk (pre-release stability concerns). The application uses **Angular standalone components** bootstrapped via `bootstrapApplication()`, aligning with modern Angular best practices.

3.2 UI & Component Libraries

Category	Libraries	Version
CSS Framework	Bootstrap	5.3.6
Angular Bootstrap	@ng-bootstrap/ng-bootstrap	18.0.0
Material Components	@angular/material, @angular/cdk	20.0.0-next.8
Icons	@ant-design/icons-angular	19.0.0
Charts	ApexCharts + ng-apexcharts	5.2.0 / 1.15.0
Calendar	angular-calendar, flatpickr, dayjs, date-fns	0.31.1 / 4.6.13
Forms	ng-select, ngx-mask, ngx-chips, ng2-date-picker, ngx-slider	Various
Rich Text	ngx-quill, ngx-editor	27.0.2 / 18.0.0
Notifications	ngx-toastr, sweetalert2, angular-notifier	19.0.0 / 11.21.2

3.3 Technology Observations

⚠ Pre-Release Dependencies: Angular 20.0.0-next.8, @angular/material 20.0.0-next.8, and @angular/cli 20.0.0-next.8 are pre-release versions. While they offer

cutting-edge features, they may contain breaking changes before stable release. This should be monitored for production readiness.

⚠ Legacy Dependencies: jQuery 3.7.1 is included as a dependency, which is unusual for a modern Angular application. This likely originates from the Mantis template's legacy code and should be evaluated for removal to reduce bundle size.

i Package Manager: Both package-lock.json (npm) and yarn.lock are present, which can lead to inconsistencies. A single package manager should be standardized.

4. Application Architecture

4.1 High-Level Architecture

The application follows a standard Angular SPA architecture with three distinct layout contexts:

- **Guest Layout** — Public pages (landing, login, register, forgot password)
- **Admin Layout** — Authenticated users with sidebar navigation, top navbar, and theme configuration panel
- **Simple Layout** — Minimal layout for component demos and contact page

4.2 Routing Architecture

The routing module defines three main route groups with lazy-loaded components:

Route Group	Layout	Auth Required	Key Paths
Guest / Public	Guest Layout	No	/, /login, /register, /forgot-password, /code-verify
Authenticated	Admin Layout	Yes (AuthGuard)	/dashboard/*, /admin/*, /pm/*, /statement/*, /helpdesk/*, /professionals/*, /chat, /calendar, /polls
Simple	Simple Layout	Varies	/components/*, /contact-us
Error	—	No	/unauthorized, ** (404)

4.3 Component Tree

```
AppComponent
├─ GuestLayoutComponent (public routes)
│  ├─ LandingComponent
│  ├─ AuthLoginComponent
│  ├─ AuthRegisterComponent
│  ├─ ForgotPasswordComponent / ResetPasswordComponent / CheckMailComponent / CodeVe
│  └─ InviteAcceptComponent
├─ AdminLayoutComponent (authenticated routes)
```

```
| |─ NavigationComponent (sidebar)
| |  └─ NavContentComponent (role-filtered menu items)
| |─ NavBarComponent
| |  └─ NavLeftComponent
| |    └─ NavRightComponent (user menu, notifications, language, fullscreen)
| |─ ConfigurationComponent (theme customization panel)
| |─ BreadcrumbComponent
| |  └─ Router Outlet (lazy-loaded feature components)
└─ SimpleLayoutComponent
    └─ Router Outlet
```

4.4 Data Flow Pattern

The application follows a unidirectional data flow pattern:

1. **User Action** → Component event handler
2. **Service Call** → HTTP request via Angular HttpClient
3. **Interceptor** → JWT token injection via BasicAuthInterceptor/JwtInterceptor
4. **API Response** → Observable stream returned to component
5. **State Update** → Component updates local state or service-based state
6. **View Update** → Angular change detection re-renders the template

5. User Roles & Access Control

5.1 Role Hierarchy

#	Role	Enum Value	Privilege Level
1	Admin	Admin	System-wide
2	Property Manager	PropertyManager	Multi-building
3	Admin Agent	AdminAgent	Multi-building
4	Property Agent	PropertyAgent	Assigned Buildings
5	Building Manager	BuildingManager	Single Building
6	Owner	Owner	Owned Apartments
7	Resident	Resident	Resident Unit
8	User	User	Basic

5.2 Role-Based Access Implementation

- **AuthGuardChild** — Route guard that checks authentication status before activating routes
- **Route Data Roles** — Routes specify data: `{ role: [...] }` for role-restricted access
- **NavigationService** — Filters sidebar menu items based on user role, showing only relevant sections
- **API-Level Authorization** — Backend services enforce authorization; the front-end primarily controls UI visibility

Note: The Role enum is defined in two locations: `role.ts` (under `_helpers`) and `role.model.ts` (under `models`). This duplication should be consolidated to a single source of truth.

6. Functional Modules

6.1 Module Overview

Module	Route	Primary Roles	Key Features
Authentication	/login, /register, /forgot-password, etc.	All (unauthenticated)	JWT login, registration, email confirmation, password reset, invitation acceptance
Dashboard	/dashboard/*	All authenticated	Default, analytics, finance dashboards with charts and KPIs
Admin Panel	/admin/*	Admin	System-wide dashboard, finance overview, building registry
Property Manager Panel	/pm/*	PropertyManager, PropertyAgent, AdminAgent	PM dashboard, building management, support agent assignment
Building Manager Panel	/statement/*	BuildingManager	Expense statement create/edit/list/details, participant management
Online Courses	/online-course/*	BuildingManager	Course management, students, teachers, pricing, settings
Membership	/membership/*	BuildingManager	Membership dashboard, list, pricing, settings
Helpdesk (Support)	/helpdesk/*	All authenticated	Ticket create/list/details, comments, assignment, status management
Professionals Directory	/professionals/*	All authenticated	Professional search, registration, approval, reviews, favorites
Chat	/chat	All authenticated	Real-time (or near-real-time) messaging

Calendar	/calendar	All authenticated	Event creation, editing, pinning, building-wide calendar
Polls	/polls	All authenticated	Poll creation, voting, results, deactivation
User Profile	/user/*	All authenticated	Profile management, account settings, user cards and lists
Customer Management	/customer	All authenticated	Customer list view

6.2 Core Business Flow: Common Expense Statement

The expense statement module is the central business feature of the application:

1. **Building Manager** creates a new statement (monthly/periodic expense allocation)
2. System **allocates expenses** across building apartments based on defined rules
3. **Notifications** are sent to residents/owners about the new statement
4. Residents/Owners can **view their allocated amount** and payment status
5. **Payments** are tracked per statement, per user, with collection rate monitoring
6. Building Manager can **edit/draft/delete** statements as needed
7. **Dashboards** display collection rates, monthly trends, and outstanding balances

6.3 Support Ticket Workflow

1. Any building member can **create a support ticket** (issue/complaint)
2. Tickets are **routed to the appropriate agent** or building manager
3. Agents can **assign, comment, and update status** (open/in-progress/resolved)
4. Users can **track their tickets** through the dashboard and inbox views
5. **Notifications** keep users informed of ticket updates

7. API Integration & Backend Services

7.1 API Architecture

Property	Value
Base URL (Dev)	http://localhost:8080
Base URL (Prod)	https://mock-data-api-nextjs.vercel.app
API Versioning	/api/v1/{resource}
Authentication	JWT Bearer Token
HTTP Client	Angular HttpClient
Dev Proxy	Angular proxy config (proxy.conf.json)

7.2 Service Inventory

Service	API Path	Key Operations
Authentication	/api/v1/auth	Login, register, confirm, password reset, user refresh
User	/api/v1/users	CRUD users, role assignment, profile image, invitations
Building	/api/v1/buildings	CRUD buildings, code-based joining, document upload, settings
Apartment	/api/v1/apartments	Batch create, list by building, update, delete
Building Member	/api/v1/building-members	List members, assign apartment, remove
Common Expense	/api/v1/expenses/statements	CRUD statements, categories, drafts
Payment	/api/v1/payments	Record, list, summary, per-building stats
Notification	/api/v1/notifications	List notifications, mark as read

Support Ticket	/api/v1/support-tickets	CRUD tickets, status, assignment, comments
Professional	/api/v1/professionals	Search, register, approve, manage, reviews, images
Calendar	/api/v1/calendar	CRUD events, pin events
Poll	/api/v1/polls	CRUD polls, voting, results
Company	/api/v1/companies	CRUD companies
Agent Management	/api/v1/agents	List agents, update building assignments
Admin Dashboard	/api/v1/admin/dashboard	Admin-level aggregated statistics
PM Dashboard	/api/v1/pm/dashboard	PM-level analytics, financial charts, collection rates
Building Dashboard	/api/v1/dashboard	Building-specific manager dashboard
User Dashboard	/api/v1/dashboard/user	User-level dashboard, history, charts, unpaid items

7.3 Integration Pattern

All services follow a consistent pattern:

- Singleton services (@Injectable({ providedIn: 'root' })))
- Base URL stored in the environment or hardcoded
- HTTP methods return typed RxJS Observable<T>
- JWT token injected via BasicAuthInterceptor / JwtInterceptor
- Standard CRUD pattern: get, create/save, update, delete

⚠ Some services use hardcoded API URLs (e.g., `http://localhost:8080`) instead of referencing `environment.apiUrl`. This should be standardized for maintainability.

8. Data Models & State Management

8.1 Data Models

The application defines **42 TypeScript interfaces and data transfer objects (DTOs)** covering:

Category	Models
Authentication	AuthenticationResponse, RegistrationRequest
Building	BuildingDTO, BuildingRequest, BuildingMeta, BuildingDocument
Apartment	ApartmentDTO, ApartmentRequest, ApartmentUpdateRequest
Members	BuildingMemberDTO, InviteRequest, UserTableDTO
Company	CompanyDTO
Expenses	CommonExpenseStatement, CommonExpenseAllocation, CommonExpenseSummaryDTO
Payments	PaymentDTO, StatementUserPaymentDTO, MonthlyAmountStatsDTO
Dashboard	AdminDashboardDTO, PmDashboardDTO, ManagerDashboardDTO, ChartDTO
Support Tickets	SupportTicket (request/response), TicketComment
Professionals	Professional model with categories, reviews, images, admin stats
Calendar	CalendarDTO
Polls	Poll model
Notifications	NotificationDTO
Navigation	NavigationItem
Pagination	PageResponse (generic paginated response)

8.2 State Management Strategy

The application does **not use a dedicated state management library** (e.g., NgRx, Akita, NgXs). Instead, it employs:

Mechanism	Usage	Examples
Angular Signals	Modern reactive state (Angular 17+)	ThemeService (dark mode, RTL), WizardStateService (persisted wizard flow)
BehaviorSubject/Subject	Reactive state with observable streams	TabNavigationService (tab index), UserDashboardService (refresh triggers)
localStorage/sessionStorage	Persistent client-side storage	JWT token, current user, building ID, wizard state, theme preferences
Service Properties	In-memory singleton state	AuthenticationService.currentUser, SharedDataService

⚠ No Dedicated State Management: As the application grows, the absence of a formal state management library may lead to challenges with:

- Cross-component state synchronization
- DevTools for debugging state changes
- Consistent state update patterns
- Performance optimization with selectors

For the current scope, the Signals + Services approach may be sufficient, but it should be revisited as features expand.

9. Internationalization & Localization

9.1 Language Support

Code	Language	Status
gr	Greek	Default
en	English	Full
fr	French	Available
ro	Romanian	Available
cn	Chinese	Available

9.2 Implementation Details

- **Library:** @ngx-translate/core with custom CustomTranslateLoader
- **Default Language:** Greek (gr), set in MantisConfig
- **Translation Format:** JSON files in src/assets/i18n/
- **Loader Strategy:** Custom loader imports JSON files at build time (not HTTP fetch)
- **Language Switching:** Via NavRightComponent language dropdown using TranslateService.use()

9.3 Business Impact

The i18n implementation is strategically important as it allows the application to serve multiple regional markets from a single codebase. The default Greek language suggests the primary target market is Greece/Cyprus, with English, French, Romanian, and Chinese supporting secondary markets.

10. Build, Deployment & DevOps

10.1 Build Pipeline

Stage	Tool	Details
Development Build	Angular CLI (ng serve)	Port 4200, proxy to localhost:8080, HMR, source maps
Production Build	Angular CLI (ng build)	Output hashing, budgets (4MB initial, 20KB component styles), AOT
Docker Build	Multi-stage Dockerfile	Node 20-alpine build → Nginx alpine serve
Docker Compose	docker-compose.yml	Port 8081:80, healthcheck, app-network bridge
Web Server	Nginx	SPA routing, 1-year asset cache, gzip

10.2 Configuration Files

File	Purpose
angular.json	Angular CLI project configuration (builder, budgets, assets, styles, scripts)
tsconfig.json / tsconfig.app.json / tsconfig.spec.json	TypeScript compilation targets (ES2022, strict mode)
eslint.config.mjs	Flat ESLint config (TypeScript + Angular template linting)
.prettierrc	Code formatting (140 char width, single quotes, no trailing commas)
.browserslistrc	Target browsers (modern Chrome, Firefox, Edge, Safari)
Dockerfile	Multi-stage Docker build for production deployment
docker-compose.yml	Container orchestration for frontend service

nginx.conf	Nginx configuration for SPA serving with caching rules
proxy.conf.json	Angular dev server proxy to backend API

10.3 CI/CD Readiness

The project is well-equipped for CI/CD with:

- Dockerized build and deployment
- Playwright config for CI (port 4200, Chromium)
- Linting (ESLint) and formatting (Prettier) configured
- No CI pipeline configuration files found (e.g., GitHub Actions, Jenkinsfile) — would need to be added

11. Quality Assurance & Testing

11.1 Testing Strategy

Type	Framework	Scope	Current Coverage
Unit Tests	Jasmine 5.1 + Karma	Component logic	Low (16 spec files)
E2E Tests	Playwright 1.61	Full user workflows	None found

11.2 Unit Test Inventory

Module	Component	Spec File Count
Authentication	Login, Register, Forgot Password, Reset Password, Check Mail, Code Verify	6
Helpdesk	Dashboard, Ticket List, Ticket Create, Ticket Details	4
Professionals	List, Approval, Manage, Register Modal, Details Modal, My Businesses	6
Total		16

11.3 Quality Assessment

⚠ **Test coverage is limited** to authentication, helpdesk, and professional modules. Core business functionality (buildings, apartments, expense statements, payments, polls, calendar, chat) has no unit tests. E2E test infrastructure is in place but no spec files have been written. This represents a significant quality risk for the core business features.

11.4 Linting & Formatting

- **ESLint:** Flat config with TypeScript and Angular-specific rules, directive/component selector enforcement
- **Prettier:** Consistent formatting with 140 char width, single quotes, no trailing commas

- No pre-commit hooks or lint-staged configuration found

12. Security Analysis

12.1 Authentication & Authorization

Control	Implementation	Effectiveness
JWT Authentication	Login endpoint returns JWT; stored in localStorage/sessionStorage	Standard
Token Interceptor	BasicAuthInterceptor / JwtInterceptor adds Bearer token to all HTTP requests	Effective
Route Guards	AuthGuardChild prevents unauthorized route access	Effective
Role-Based Access	Route data roles + NavigationService filtering	Good
Token Persistence	localStorage (remember me) vs sessionStorage (session only)	Configurable
XSS Protection	Angular's built-in sanitization	Standard

12.2 Security Considerations

- **JWT Storage:** Tokens stored in browser storage (localStorage/sessionStorage) which is susceptible to XSS attacks. Consider httpOnly cookies for production.
- **Role Duplication:** Role enum defined in two places increases risk of inconsistencies
- **reCAPTCHA:** ng-recaptcha is included, suggesting CAPTCHA protection for forms
- **Backend Authorization:** The front-end role checks are primarily UX-level; real authorization must be enforced on the backend

⚠ **Environment Exposure:** The production API URL (`https://mock-data-api-nextjs.vercel.app`) is compiled into the front-end bundle. Ensure this is acceptable from a security standpoint.

13. Performance Considerations

13.1 Bundle Size

Metric	Budget	Notes
Initial Bundle	4 MB (warning) / 5 MB (error)	Angular + Bootstrap + 78 dependencies is sizable
Component Styles	20 KB	Per-component style budget

13.2 Performance Factors

Factor	Impact	Mitigation
78 Runtime Dependencies	Medium	Tree-shaking via Angular CLI; evaluate unused deps
jQuery Inclusion	Low-Medium	Evaluate removal; likely unused in Angular context
Multiple CSS Frameworks	Low	Bootstrap + Material CDK; some overlap
AOT Compilation	Positive	Enabled by default in Angular production builds
Lazy-Loaded Routes	Positive	Feature modules are lazy-loaded
CDK Overlay	Positive	Efficient overlay rendering from Angular Material CDK
Nginx Caching	Positive	1-year cache for static assets, gzip compression
Custom Fonts	Low-Medium	4 font families available; variable fonts would reduce payload

14. Risk Assessment

14.1 Risk Matrix

#	Risk	Severity	Likelihood	Mitigation
R1	Pre-release Angular 20 (next.8) — potential breaking changes before stable	High	Medium	Monitor Angular changelog; plan migration path to stable; run tests frequently
R2	Limited test coverage on core business modules (statements, payments, buildings)	High	High	Prioritize unit tests for expense statements, payments, building management
R3	No E2E tests for critical user workflows	High	High	Implement Playwright E2E tests for core flows: login → statement → payment
R4	Duplicate Role enum definitions may lead to authorization inconsistencies	Medium	Medium	Consolidate to single source of truth; add lint rule
R5	Mixed package manager (npm + yarn lock files) may cause CI/build conflicts	Medium	Medium	Standardize on one package manager; remove the other lock file
R6	No formal documentation or README — onboarding friction for new developers	Medium	High	Create README with setup instructions, architecture overview, and contribution guide
R7	jQuery dependency in Angular app — unnecessary bundle bloat and technical debt	Low	Low	Audit jQuery usage; remove if unused
R8	Hardcoded API URLs in some services instead of environment variables	Medium	Medium	Refactor to use environment.apiUrl

				consistently across all services
R9	No dedicated state management — potential scaling issues as features grow	Medium	Low	Evaluate need for NgRx or similar; cross-bridge when state complexity increases
R10	No CI/CD pipeline configuration in repository	Medium	Medium	Add GitHub Actions or GitLab CI configuration for automated testing and deployment

15. Recommendations

15.1 Immediate (Short-Term)

1. **Standardize package manager** — Remove either `yarn.lock` or `package-lock.json` to avoid dependency conflicts
2. **Consolidate Role enum** — Merge `role.ts` and `role.model.ts` into a single source of truth
3. **Create README.md** — Document setup instructions, architecture, environment variables, and deployment process
4. **Audit jQuery usage** — Determine if jQuery is actively used or can be safely removed to reduce bundle size
5. **Standardize API URLs** — Refactor all services to use `environment.apiUrl` instead of hardcoded values

15.2 Medium-Term (1-3 Months)

1. **Expand test coverage** — Prioritize unit tests for: expense statements CRUD, payment processing, building & apartment management, user role transitions
2. **Implement E2E tests** — Use Playwright for critical user journeys: registration → building creation → statement creation → payment tracking
3. **Add CI/CD pipeline** — GitHub Actions workflow for lint → test → build → deploy stages
4. **Migrate to stable Angular** — Once Angular 20 reaches stable, migrate the project to eliminate pre-release risk
5. **Remove unused dependencies** — Conduct a full audit of the 78 runtime dependencies and remove any unused packages

15.3 Long-Term (3-6 Months)

1. **Evaluate state management** — If state complexity grows, consider adopting Angular Signals fully or introducing a lightweight state management solution
2. **Performance optimization** — Implement lazy loading for images, consider variable fonts, audit bundle size with `source-map-explorer`
3. **Accessibility (a11y)** — Audit components for WCAG compliance; ensure keyboard navigation and screen reader support

4. **PWA readiness** — Evaluate adding service worker for offline capabilities and improved loading performance
5. **Monitoring & analytics** — Integrate application performance monitoring (e.g., Sentry, Datadog) for production observability

15.4 Business Strategy Recommendations

Area	Recommendation	Expected Benefit
Market Positioning	Leverage Greek language default for Greek/Cypriot market expansion	Strong local market fit
Monetization	Membership module suggests tiered pricing model — refine and highlight	Recurring revenue stream
Professional Directory	Monetize as a lead-generation platform for property service providers	Additional revenue channel
Multi-Building Support	Property Manager role supports multi-building portfolios — key B2B selling point	Enterprise adoption
White-Label Readiness	Extensive theme customization enables white-labeling for property management companies	Partnership & OEM opportunities
Internationalization	5 languages support expansion into French, Romanian, and Chinese markets	Geographic scalability

